

WEB-ПРОГРАММИРОВАНИЕ

ПРОЕКТНО-ТЕХНОЛОГИЧЕСКАЯ ПРАКТИКА

СОБОРОВА ВИКТОРИЯ ВИКТОРОВНА, каф. ИУ9

Веб-программирование — раздел программирования, ориентированный на разработку веб-приложений(программ, обеспечивающих функционирование динамических сайтов)

Языки веб-программирования — это языки, которые в основном предназначены для работы с веб-технологиями. Языки веб-программирования можно условно разделить на две пересекающиеся группы:

FRONT-END — клиентские языки (html, css, javascript)

BACK-END — серверные языки программирования (php, sql, python, go, ruby, java и т.п.)

Протоколы передачи данных

FTP — для передачи файлов

SMTP — для отправки писем

IMAP — для получения писем

IRC — для чата

HTTP — для просмотра гипертекстовых документов

FRONT-END — разработка видимого для пользователя интерфейса и функциональности элементов, с которыми он может взаимодействовать

— текстовые и графические блоки

— шрифты

— модальные окна, карусель, анимация и т.п.

BACK-END — не видимая для пользователя область, взаимодействующая с сервером

— базы данных

— почтовый сервер

— рассылка писем

1. Техническое задание (ТЗ) — цель отдания ресурса, объем сайта, функциональность, структура сайта
2. Макетирование (PSD) — это графический файл, представляющий собой многослойный рисунок, элементами которого являются мелкие картинки
3. Верстка — (верстка структуры сайта, блоки и элементы (по макету)
4. Дизайн — подключение таблиц каскадных стилей для внешнего оформления веб-страниц (по макету)
5. Функциональность — прописывается функциональность сайта (анимирование, сервисы, базы данных и т.п.)
6. Тестирование/отладка — всевозможные проверки, юзабилити-тестирование; имеющие место ошибки подлежат анализу и исправлению
7. Размещение — размещения файлов на сервере и производство необходимых настроек
8. Внутренняя SEO-оптимизация — адаптации контента для поисковых систем (подбор ключевых слов)
9. Внешняя SEO-оптимизация — раскрутка сайта; осуществляется путём построения структуры входящих ссылок
10. Сдача проекта — подписания документов о сдаче проекта и разъясняются все нюансы, касающиеся деятельности в административной зоне сайта

- **FRONT-END**

- HTML
- CSS
- ВАЛИДНОСТЬ
- АДАПТИВНОСТЬ
- КРОССБРАУЗЕРНОСТЬ
- SASS
- МЕТОДОЛОГИЯ БЭМ
- JS
- BOOTSTRAP/REACT/ANGULAR/TYPESCRIPT/COFFEESCRIPT
- WEBPACK/GULP
- NODE.JS
- PWA

- **BACK-END**

- MYSQL
- PHP
- PHPMYADMIN
- LARAVEL
- APACHE

- **DOCKER**

- IMAGES
- CONTAINERS
- VOLUMES
- DOCKER—COMPOSE
- DOCKER—MACHINE

- **APP**

- TERMINAL
- HOMEBREW/NPM
- VSCODE/BRACKETS
- PLUGINS
- DOCKER DESKTOP/GITHUB DESKTOP
- FIREFOX DEVELOPER EDITION
- LIGHTHOUSE
- FTP-CLIENT/SSH/SSL

FRONT-END

HTML — СТРУКТУРА

HTML5 — язык разметки гипертекстовых документов, состоит из тегов и атрибутов; набор тегов представляет собой блоки и их элементы

```
<!DOCTYPE>
```

```
<!-- Комментарий -->
```

```
<html>
```

```
<head>
```

```
    <meta>
```

```
    <title></title>
```

```
    <link>
```

```
    <style></style>
```

```
    <script></script>
```

```
</head>
```

```
<body>
```

```
    <header></header>
```

```
    <main></main>
```

```
    <footer></footer>
```

```
    <script></script>
```

```
</body>
```

```
</html>
```

FRONT-END

HTML — ТЕГИ

<section>

<div>

<h1> <h2> <h3> <h4> <h5> <h6>

<p>

<a>

<nav>

<menu>

<select> <optgroup> <option>

<form> <input>

<table> <th> <tr> <td>

<iframe>

<video>

<map>

<hr>

Типы тегов

	HTML5
	Блочные элементы
	Строчные элементы
	Универсальные элементы
	Нестандартные теги
	Осуждаемые теги
	Видео
	Документ
	Звук
	Изображения
	Объекты
	Скрипты
	Списки
	Ссылки
	Таблицы
	Текст
	Форматирование
	Формы
	Фреймы

FRONT-END

HTML — АТРИБУТЫ

Атрибуты — дополнительная информация к тегам; пишутся внутри открывающего тега; со значением или без значения

Типы атрибутов — универсальные, уникальные, события

Порядок атрибутов в тегах — универсальные указываются первыми(class, id); уникальные вторыми (action); события указываются последними (onClick, disabled)

```
<input class="search-form__input-text" type="text"
placeholder="Name" disabled>
```


FRONT-END

HTML — АТРИБУТЫ

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
5      <meta name="viewport" content="width=device-width, user-scalable=no, initial-
6      scale=1.0, maximum-scale=1.0, minimum-scale=1.0">
7      <title>Document</title>
8      <link rel="stylesheet" href="css/style.css">
9      <style></style>
10     <script></script>
11 </head>
12 <body>
13     <header>
14         <ul>
15             <li><a href="#link1">Ссылка 1</a></li>
16             <li><a href="#link2">Ссылка 2</a></li>
17             <li><a href="#link3">Ссылка 3</a></li>
18         </ul>
19     </header>
20     <main>
21         <section class="section-title">
22             <a name="link1"></a>
23             <div class="block-title">
24                 <h1 class="title">Hello World!</h1>
25             </div>
26         </section>
27         <section class="section-about">
28             <a name="link2"></a>
29             <div class="block-description">
30                 <p class="description">Lorem ipsum dolor sit amet, consectetur
31                 adipiscing elit. Molestias, ex.</p>
32             </div>
33         </section>
34     </main>
35     <footer>
36         <a name="link3"></a>
37         <p>All Rights Reserved</p>
38     </footer>
39     <!--      Java Script-->
40 </body>
41 </html>
```

FRONT-END

CSS

Таблицы каскадных стилей для оформления внешнего вида блоков и элементов

```
/*Комментарий*/
```

```
/*Тег*/
```

```
div{  
    свойство: значение;  
}
```

```
/*Класс*/
```

```
.class{  
    свойство: значение;  
}
```

```
/*Идентификатор*/
```

```
#id{  
    свойство: значение;  
}
```

FRONT-END

CSS

Файл – index.html

```
<div class="text-block">
  <h1>
    Заголовок размером h1
  </h1>
  <p>
    Текстовый параграф
  </p>
</div>
```

Файл – style.css

```
/*глобальный указатель*/
div{
  width: 100%;
  height: auto;
}
/*с указанием элемента и его названия*/
div.text-block{
  width: 100%;
  height: auto;
}
/*только название элемента*/
.text-block{
  width: 100%;
  height: auto;
}
/*с явным указанием пути от родительского элемента*/
.text-block > h1{
  color: red;
}
/*с неявным указанием пути от родительского элемента*/
.text-block h1{
  color: red;
}
```

FRONT-END

ВАЛИДНОСТЬ

Валидность HTML-кода — соответствие кода сайта стандартам, описанным Консоциумом Всемирной Паутины (W3C)

Проверка HTML-кода сайта на валидность осуществляется с помощью специального инструмента. Проверить код можно, указав URL сайта, загрузив часть кода или файл с ним

HTML-валидатор произведет несколько проверок загруженного кода:

- Валидация синтаксиса — проверка на наличие синтаксических ошибок
- Проверка вложенности тегов
- Валидация DTD — проверка соответствия кода указанному Document Type Definition. Она включает проверку названий тегов, атрибутов, и встраивания тегов
- Проверка на посторонние элементы — проверка выявляет все, что есть в коде, но отсутствует в DTD. Например, пользовательские теги и атрибуты.

Отчет, который предоставляет валидатор от W3C, содержит:

- Список ошибок и предупреждений, с указанием критичности
- Строка тега с ошибкой
- Рекомендации по исправлению

FRONT-END

АДАПТИВНОСТЬ

Адаптивность сайта – это способность веб-ресурса корректно отображаться на устройствах с разными аппаратными и программными характеристиками

- мета-тег масштабирования
- адаптивные размерности элементов
- медиа-запросы для различных устройств
- систему двумерного макета

FRONT-END

АДАПТИВНОСТЬ — Метатег viewport

Для управления разметкой в мобильных браузерах используется метатег viewport. Изначально данный тег был представлен разработчиками Apple для браузера Safari на iOS. Мобильные браузеры отображают страницы в виртуальном окне просмотра, которое обычно шире, чем экран устройства. С помощью метатега viewport можно контролировать размер окна просмотра и масштаб

Страницы, адаптированные для просмотра на разных типах устройств, должны содержать в разделе <head> метатег viewport

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Свойство width определяет виртуальную ширину окна просмотра, значение device-width — физическую ширину устройства

При первой загрузке страницы свойство initial-scale управляет начальным уровнем масштабирования, initial-scale=1 означает, что 1 пиксель окна просмотра = 1 пиксель CSS

FRONT-END

АДАПТИВНОСТЬ — Адаптивные размерности

%

vw

vh

vmin

vmax

FRONT-END

АДАПТИВНОСТЬ — Устройства

При составлении медиазапросов нужно ориентироваться на так называемые переломные (контрольные) точки дизайна, т.е. такие значения ширины области просмотра, в которых дизайн сайта существенно меняется

```
/* Смартфоны (вертикальная и горизонтальная  
ориентация) */
```

```
@media only screen and (min-width: 320px) and (max-  
width: 480px) and (orientation: landscape) {
```

```
    /* СТИЛИ */
```

```
}
```


FRONT-END

АДАПТИВНОСТЬ — Модуль CSS Grid Layout

Система двумерного макета, оптимизированного для дизайна пользовательского интерфейса. Главная идея, лежащая в основе макета сетки, заключается в разделении веб-страницы на столбцы и строки. В образовавшиеся области сетки можно помещать элементы сетки, а управлять их размерами и расположением можно с помощью специальных свойств модуля

Сетка

```
<!-- html разметка -->
```

```
<div class="grid-container">
```

```
    <div>...</div>
```

```
    <div>...</div>
```

```
</div>
```

Свойства

```
/* CSS СВОЙСТВА */
```

```
.grid-container {
```

```
    display: grid;
```

```
    grid-template-columns: [line1]50% [line2]50%;
```

```
}
```

FRONT-END

КРОССБРАУЗЕРНОСТЬ

Это способность веб-ресурса отображаться одинаково хорошо во всех популярных браузерах без перебоев в функционировании и ошибок в верстке, с одинаково корректной читабельностью контента. Это очень важный показатель как для поисковых систем, так и пользовательской аудитории

Кроссбраузерными считаются сайты, которые хорошо работают на:

- Google Chrome и производных обозревателях (типа Яндекс.Браузера, Chromium, Opera с 15 версии и т.д.)
- Microsoft Edge
- Mozilla Firefox
- Safari (iOS и macOS)

FRONT-END

КРОССБРАУЗЕРНОСТЬ

Использовать универсальные элементы и/или применить вендорные префиксы

Префиксы — приставки к названиям CSS-свойств, используемые определенными браузерами. Они позволяют изменять отображение в конкретном обозревателе

- `-moz-` для Mozilla Firefox
- `-ms-` используется в Internet Explorer и Microsoft Edge
- `-webkit-` для Safari, Google Chrome
- `-o-` используется в Opera

```
.block {  
  
    transform: 90deg;  
  
    -o-transform: 90deg;  
  
    transition: 3s;  
  
    -o-transition: 3s;  
  
}
```

FRONT-END

SASS

Syntactically Awesome Style Sheets

- <https://sass-lang.com/>

Sass — это метаязык (язык для описания другого языка), который упрощает и ускоряет написание CSS-кода. Его часто называют препроцессором CSS

Препроцессор — это компьютерная программа, принимающая данные на входе и выдающая данные, предназначенные для входа другой программы

Основная задача препроцессора — это предоставление удобных синтаксических конструкций для разработчика, чтобы упростить, и тем самым, ускорить разработку и поддержку стилей в проектах. CSS препроцессоры преобразуют код, написанный с использованием препроцессорного языка, в чистый и валидный CSS-код

FRONT-END

SASS — КОДИРОВКА

Чтобы избежать потенциальных проблем с кодировкой символов, крайне рекомендуется использовать кодировку UTF-8 в основной таблице стилей, используя директиву `@charset`. Убедитесь, что она идёт первой строкой в таблице стилей и перед ней ничего нет.

```
@charset 'utf-8';
```

FRONT-END

SASS — ПЕРЕМЕННЫЕ

Переменные работают по такому же принципу, как и в любом языке программирования. Объявляя переменную, мы храним в ней какое-либо значение, которое обычно встречается в CSS в виде цвета, шрифта или целого набора свойств, например для box-shadow.

Идея в том, что данный подход упрощает процесс повторного использования переменных, а также мы можем быстро изменить значение конкретной переменной там, где мы её объявляем, вместо повсеместного перепечатывания кода.

```
$title-font: normal 24px/1.5 'Open Sans', sans-serif
$cool-red: #F44336
$box-shadow-bottom-only: 0 2px 1px 0 rgba(0, 0, 0, 0.2)

h1.title
  font: $title-font
  color: $cool-red

div.container
  color: $cool-red
  background: #fff
  width: 100%
  box-shadow: $box-shadow-bottom-only
```

FRONT-END

SASS — МИКСИНЫ Mixin (примеси)

Mixin можно также представить как класс-конструктор в языке программирования: вы используете ряд свойств из CSS, создавая отдельный объект, который потом используете где хотите, задавая разные значения его свойствам.

```
@mixin square($size, $color)
  width: $size
  height: $size
  background-color: $color

.small-blue-square
  @include square(20px, rgb(0,0,255))

.big-red-square
  @include square(300px, rgb(255,0,0))
```


FRONT-END

SASS — МИКСИНЫ Mixin (примеси)

Ещё один способ упростить себе работу с помощью Mixin - использование его в местах, где требуются префиксы для адаптации под разные браузеры.

```
@mixin transform-tilt()  
  $tilt: rotate(15deg)  
  -webkit-transform: $tilt  
  -ms-transform: $tilt  
  transform: $tilt  
  
.frame:hover  
  @include transform-tilt
```


FRONT-END

SASS — ЭКСТЕНДЫ Extend

Следующая особенность, на которую мы взглянем, будет `@extend`, она позволяет вам наследовать CSS-свойства одного селектора от другого. Принцип работы напоминает `Mixin`, но `Extend`, как правило, используется для того, чтобы создать логическую связь между элементами страницы.

`Extend` используется, когда нам, к примеру, нужно два похожих элемента, которые имеют некоторые отличия. Например, давайте возьмём две кнопки: согласие и отмена.

Если вы взгляните на CSS код, то заметите, что Sass скомбинировал селекторы вместо повторения одних и тех же строк несколько раз в коде.

```
.dialog-button
  box-sizing: border-box
  color: #ffffff
  box-shadow: 0 1px 1px 0 rgba(0, 0, 0, 0.12)
  padding: 12px 40px
  cursor: pointer

.confirm
  @extend .dialog-button
  background-color: #87bae1
  float: left

.cancel
  @extend .dialog-button
  background-color: #e4749e
  float: right
```

FRONT-END

SASS — ВЛОЖЕННЫЕ КОНСТРУКЦИИ

Как известно, в HTML, как правило, программист пишет код по принципу "гнездования". Иными словами, блоки кода находятся в других блоках кода и содержат вложенные блоки кода. CSS же в этом плане представляет собой полнейший хаос. Если для вас это проблема, Sass может помочь вам в организации кода.

```
ul
  list-style: none
  li
    padding: 15px
    display: inline-block
    a
      text-decoration: none
      font-size: 16px
      color: #444
```

FRONT-END

SASS — ОПЕРАЦИИ

Вы можете выполнять различные математические операции прямо в коде, что значительно упрощает работу в некоторых случаях.

Хоть сейчас и обычный CSS предлагает те же возможности, правда в форме `calc()`, альтернативный вариант на Sass быстрее в написании, имеет операцию `mod (%)` и применяется на более широком спектре типов данных (цвета, строки).

```
$width: 800px

.container
  width: $width

.column-half
  width: $width / 2

.column-fifth
  width: $width / 5
```

FRONT-END

SASS — ФУНКЦИИ

В Sass имеется целый ряд встроенных функций разного рода. К примеру, функции для операций со строками, цветами или выполняющие различные математические операции вроде `random()` или `round()`.

Чтобы было нагляднее, представим функцию `darken($color, $amount)`, которая, как понятно из названия, затемняет или применяет `hover`.

```
$awesome-blue: #2196F3
```

```
a
  padding: 10px 15px
  background-color: $awesome-blue
```

```
a:hover
  background-color: darken($awesome-blue, 10%)
```

FRONT-END

SASS — КОМПИЛИРОВАНИЕ

1. Команда в терминале

```
node-sass input.sass output.css
```

2. Специализированные плагины

3. Онлайн компиляторы

<https://beautifytools.com/sass-compiler.php>

FRONT-END

МЕТОДОЛОГИЯ БЭМ

БЭМ (Блок, Элемент, Модификатор) — компонентный подход к веб-разработке. В его основе лежит принцип разделения интерфейса на независимые блоки. Он позволяет легко и быстро разрабатывать интерфейсы любой сложности и повторно использовать существующий код, избегая «Copy-Paste».

- Типичная верстка в Яндексе (2005)
- Зарождение основ методологии (2006)
- Зачатки общепортального фреймворка (2006)
- Верстка независимыми блоками (2007)
- Общепортальный фреймворк — Лего (2008)
- Общепортальный фреймворк — Лего 1.2 (2008)
- Лего 2.0. Появление БЭМ (2009)
- БЭМ и open source (2010)

FRONT-END

МЕТОДОЛОГИЯ БЭМ — БЛОК

Функционально независимый компонент страницы, который может быть повторно использован.

В HTML блоки представлены атрибутом class

Особенности:

- Название блока характеризует смысл («что это?» — «меню»: menu, «кнопка»: button), а не состояние («какой, как выглядит?» — «красный»: red, «большой»: big)
- Блок не должен влиять на свое окружение, т. е. блоку не следует задавать внешнюю геометрию (в виде отступов, границ, влияющих на размеры) и позиционирование
- В CSS по БЭМ также не рекомендуется использовать селекторы по тегам или id
- Блоки можно вкладывать друг в друга
- Допустима любая вложенность блоков

Таким образом обеспечивается независимость, при которой возможно повторное использование или перенос блоков с места на место

FRONT-END

МЕТОДОЛОГИЯ БЭМ — БЛОК

НАЗВАНИЯ БЛОКОВ

```
<!-- Верно. Семантически осмысленный блок `error` -->  
<div class="error"></div>  
<!-- Неверно. Описывается внешний вид -->  
<div class="red-text"></div>
```

ВЛОЖЕННОСТЬ БЛОКОВ

```
<!-- Блок `header` -->  
<header class="header">  
  <!-- Вложенный блок `logo` -->  
  <div class="logo"></div>  
  
  <!-- Вложенный блок `search-form` -->  
  <form class="search-form"></form>  
</header>
```


FRONT-END

МЕТОДОЛОГИЯ БЭМ — ЭЛЕМЕНТ

Составная часть блока, которая не может использоваться в отрыве от него

Особенности:

- Название элемента характеризует смысл («что это?» — «пункт»: `item`, «текст»: `text`), а не состояние («какой, как выглядит?» — «красный»: `red`, «большой»: `big`)
- Структура полного имени элемента соответствует схеме: имя-блока__имя-элемента. Имя элемента отделяется от имени блока двумя подчеркиваниями (`__`)
- Элементы можно вкладывать друг в друга
- Допустима любая вложенность элементов
- Элемент — всегда часть блока, а не другого элемента. Это означает, что в названии элементов нельзя прописывать иерархию вида `block__elem1__elem2`
- Блок может иметь вложенную структуру элементов в DOM-дереве
- Структура блока меняется, а правила для элементов и их названия остаются прежними
- Элемент — всегда часть блока и не должен использоваться отдельно от него

FRONT-END

МЕТОДОЛОГИЯ БЭМ — ЭЛЕМЕНТ

ВЛОЖЕННОСТЬ ЭЛЕМЕНТОВ

```
<!-- Блок `search-form` -->
<form class="search-form">
  <!-- Элемент `input` блока `search-form` -->
  <input class="search-form__input">

  <!-- Элемент `button` блока `search-form` -->
  <button class="search-form__button">Найти</button>
</form>
```

ДОМ ДЕРЕВО

```
<div class="block">
  <div class="block__elem1">
    <div class="block__elem2">
      <div class="block__elem3"></div>
    </div>
  </div>
</div>
<div class="block">
  <div class="block__elem1">
    <div class="block__elem2"></div>
  </div>

  <div class="block__elem3"></div>
</div>
```

FRONT-END

МЕТОДОЛОГИЯ БЭМ — МОДИФИКАТОР

Сущность, определяющая внешний вид, состояние или поведение блока либо элемента

Особенности:

- Название модификатора характеризует внешний вид («какой размер?», «какая тема?» и т. п. — «размер»: `size_s`, «тема»: `theme_islands`), состояние («чем отличается от прочих?» — «отключен»: `disabled`, «фокусированный»: `focused`) и поведение («как ведет себя?», «как взаимодействует с пользователем?» — «направление»: `directions_left-top`)
- Имя модификатора отделяется от имени блока или элемента одним подчеркиванием (`_`)
- С точки зрения БЭМ-методологии модификатор не может использоваться в отрыве от модифицируемого блока или элемента. Модификатор должен изменять вид, поведение или состояние сущности, а не заменять ее

FRONT-END

МЕТОДОЛОГИЯ БЭМ — МОДИФИКАТОР — БУЛЕВЫЙ

- Используют, когда важно только наличие или отсутствие модификатора, а его значение несущественно. Например, «отключен»: disabled. Считается, что при наличии булевого модификатора у сущности его значение равно true
- Структура полного имени модификатора соответствует схеме:
 - имя-блока_имя-модификатора
 - имя-блока__имя-элемента_имя-модификатора

```
<!-- Блок `search-form` имеет булевый модификатор `focused` -->
<form class="search-form search-form_focused">
  <input class="search-form__input">

  <!-- Элемент `button` имеет булевый модификатор `disabled` -->
  <button class="search-form__button search-
form__button_disabled">Найти</button>
</form>
```

FRONT-END

МЕТОДОЛОГИЯ БЭМ — МОДИФИКАТОР — КЛЮЧ-ЗНАЧЕНИЕ

- Используют, когда важно значение модификатора. Например, «меню с темой оформления islands»: menu_theme_islands.
- Структура полного имени модификатора соответствует схеме:
 - имя-блока_имя-модификатора_значение-модификатора;
 - имя-блока__имя-элемента_имя-модификатора_значение-модификатора

```
<!-- Блок `search-form` имеет модификатор `theme` со значением  
`islands` -->  
<form class="search-form search-form_theme_islands">  
  <input class="search-form__input">  
  
  <!-- Элемент `button` имеет модификатор `size` со значением `m` -->  
  <button class="search-form__button search-  
form__button_size_m">Найти</button>  
</form>
```

FRONT-END

МЕТОДОЛОГИЯ БЭМ — МОДИФИКАТОР — МИКС

Прием, позволяющий использовать разные БЭМ-сущности на одном DOM-узле.

Миксы позволяют:

- совмещать поведение и стили нескольких сущностей без дублирования кода
- создавать семантически новые компоненты интерфейса на основе имеющихся

Такой подход позволяет нам задать внешнюю геометрию и позиционирование в элементе `header__search-form`, а сам блок `search-form` оставить универсальным. Таким образом, блок можно использовать в любом другом окружении, потому что он не специфицирует никакие отступы. Это позволяет нам говорить о его независимости.

```
<!-- Блок `header` -->
<div class=«header">
  <!-- К блоку `search-form` примиксован элемент `search-form` блока
  `header`-->
  <div class="search-form header__search-form"></div>
</div>
```

FRONT-END

МЕТОДОЛОГИЯ БЭМ — ФАЙЛОВАЯ СТРУКТУРА

Принятый в методологии БЭМ компонентный подход применяется и к организации проектов в файловой структуре. Реализации блоков, элементов и модификаторов делятся на независимые файлы-технологии, что позволяет нам подключать их опционально.

Особенности:

- Один блок — одна директория.
- Имена блока и его директории совпадают. Например, блок `header` — директория `header/`, блок `menu` — директория `menu/`.
- Реализация блока разделяется на отдельные файлы-технологии. Например, `header.css`, `header.js`.
- Директория блока является корневой для поддиректорий соответствующих ему элементов и модификаторов.
- Имена директорий элементов начинаются с двойного подчеркивания (`__`). Например, `header/__logo/`, `menu/__item/`.
- Имена директорий модификаторов начинаются с одинарного подчеркивания (`_`). Например, `header/_fixed/`, `menu/_theme_islands/`.
- Реализации элементов и модификаторов разделяются на отдельные файлы-технологии. Например, `header__input.js`, `header_theme_islands.css`.

FRONT-END

МЕТОДОЛОГИЯ БЭМ — ФАЙЛОВАЯ СТРУКТУРА

search-form/	# Директория блока search-form
__input/	# Поддиректория элемента search-
form__input	
search-form__input.css	# Реализация элемента search-form__input
	# в технологии CSS
search-form__input.js	# Реализация элемента search-form__input
	# в технологии JavaScript
__button/	# Поддиректория элемента search-
form__button	
search-form__button.css	
search-form__button.js	
_theme/	# Поддиректория модификатора
	# search-form_theme
search-form_theme_islands.css	# Реализация блока search-form, имеющего
	# модификатор theme со значением islands
	# в технологии CSS
search-form_theme_lite.css	# Реализация блока search-form, имеющего
	# модификатор theme со значением lite
	# в технологии CSS
search-form.css	# Реализация блока search-form
	# в технологии CSS
search-form.js	# Реализация блока search-form
	# в технологии JavaScript

FRONT-END

МЕТОДОЛОГИЯ БЭМ — ТЕХНОЛОГИЯ РЕАЛИЗАЦИИ

Технология, которая используется для реализации блока.

Блоки могут быть реализованы в одной или нескольких технологиях, например:

- поведение — JavaScript, CoffeeScript;
- внешний вид — CSS, Stylus, Sass;
- шаблоны — Pug, Handlebars, XSL, BEMHTML, ВН;
- документация — Markdown, Wiki, XML.

Например, если внешний вид блока задан с помощью CSS, это означает, что блок реализован в технологии CSS. А если документация к блоку написана в формате Markdown — блок реализован в технологии Markdown.

FRONT-END

МЕТОДОЛОГИЯ БЭМ — СОГЛАШЕНИЕ ПО ИМЕНОВАНИЮ

Имя БЭМ-сущности уникально. Во всех технологиях (CSS, JavaScript, HTML) одна и та же БЭМ-сущность всегда называется одинаково. Основная идея соглашения по именованию — вложить смысл в имена и сделать их максимально информативными для разработчика.

Можно сравнить одно и тоже имя CSS-селектора, написанное разными способами:

- `menuitemvisible`
- `menu-item-visible`
- `menuItemVisible`

Чтобы понять смысл первого имени, нужно вчитаться в каждое слово. В последних двух примерах имя явно разделяется на логические части. Но ни одно из имен пока не помогает точно определить, что `menu` — это блок, `item` — элемент, а `visible` — модификатор. Чтобы имена сущностей были однозначными и понятными, в БЭМ были разработаны правила формирования имен БЭМ-сущностей.

FRONT-END

МЕТОДОЛОГИЯ БЭМ — СОГЛАШЕНИЕ ПО ИМЕНОВАНИЮ — ПРАВИЛА ФОРМИРОВАНИЯ ИМЕН

`block-name__elem-name_mod-name_mod-val`

- Имена записываются латиницей в нижнем регистре
- Для разделения слов в именах используется дефис (-)
- Имя блока задает пространство имен для его элементов и модификаторов
- Имя элемента отделяется от имени блока двумя подчеркиваниями (__)
- Имя модификатора отделяется от имени блока или элемента одним подчеркиванием (_)
- Значение модификатора отделяется от имени модификатора одним подчеркиванием (_)
- Значение булевых модификаторов в имени не указывается

Описанные выше правила формирования имен — это классическая схема именования БЭМ-сущностей. Все инструменты БЭМ по умолчанию настроены на классическую схему

Существуют альтернативные схемы именования, которые активно используются в БЭМ-сообществе. Чтобы во всех технологиях применять одинаковые имена, созданные по альтернативным схемам, используйте инструмент `bem-naming`. По умолчанию `bem-naming` содержит настройки соглашения по именованию, предложенного методологией, но позволяет добавлять правила для применения альтернативных схем

block-name__elem-name--mod-name--mod-val

- Имена записываются латиницей в нижнем регистре
- Для разделения слов в именах БЭМ-сущностей используется дефис (-)
- Имя элемента отделяется от имени блока двумя подчеркиваниями (__)
- Булевые модификаторы отделяются от имени блока или элемента двумя дефисами (--)
- Значение модификатора отделяется от его имени двумя дефисами (—)

blockName__elemName_modName_modVal

- Имена записываются латиницей
- Каждое слово внутри имени пишется с заглавной буквы
- Разделители элементов и модификаторов совпадают с классической схемой

BlockName-ElemName_modName_modVal

- Имена записываются латиницей
- Имена блоков и элементов пишутся с заглавной буквы. Имена модификаторов — со строчной
- Каждое слово внутри имени пишется с заглавной буквы
- Имя элемента отделяется от имени блока одним дефисом (-)
- Разделители имени и значения модификаторов совпадают с классической схемой

`_available`

- Имена записываются латиницей.
- Имя блока или элемента перед модификатором не указывается.

Такая схема именования ограничивает использование миксов, так как не дает возможности определить, к какому блоку или элементу относится модификатор

Можно создать собственную схему именования БЭМ-сущностей. Важно, чтобы разделители в новой схеме давали возможность на программном уровне отделять блоки от элементов и модификаторов

FRONT-END

МЕТОДОЛОГИЯ БЭМ — ПЛАТФОРМА

Чтобы генерировать HTML-код автоматически и иметь возможность внести изменения в реализацию блока в одном файле и применить ко всем экземплярам блока в разметке, в БЭМ используются шаблоны.

Шаблоны — это технология реализации блока, результатом работы которой является HTML этого блока. С помощью шаблонов текущий HTML-элемент может быть подменен на другой или дополнен новыми.

В БЭМ-платформе разработана технология для создания шаблонов — BEMHTML. Все примеры в этом разделе приведены с использованием этого шаблонизатора.

Шаблоны в БЭМ пишутся декларативно. Это позволяет применять к ним основные принципы методологии:

- Использовать термины блоков, элементов и модификаторов
- Разделять код на части
- Использовать уровни переопределения

FRONT-END

МЕТОДОЛОГИЯ БЭМ — СБОРКА

В БЭМ-проекте код разделяется на отдельные файлы (также называемые исходными файлами). Чтобы объединить исходные файлы в один (например, все CSS-файлы в `project.css`, все JS-файлы в `project.js` и т. п.), используется сборка. Файлы, полученные в результате сборки, в БЭМ-методологии принято называть бандлами

Сборка решает следующие задачи:

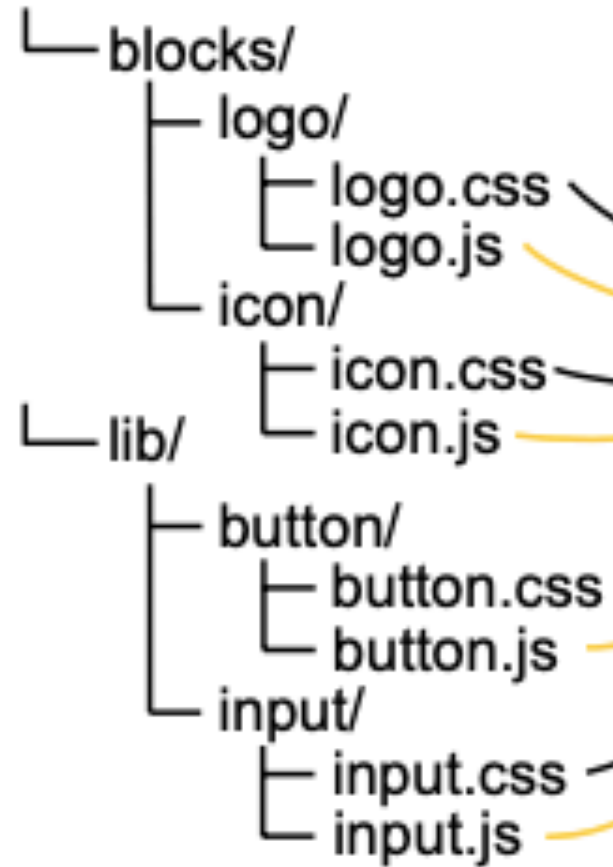
- Объединяет исходные файлы, разложенные по файловой структуре проекта.
- Подключает в проект только необходимые блоки, элементы и модификаторы (БЭМ-сущности)
- Учитывает порядок подключения
- Обработывает код исходных файлов в процессе сборки (например, компилирует LESS-код в CSS-код)

FRONT-END

МЕТОДОЛОГИЯ БЭМ — СБОРКА

Source files

project/



logo.css
logo.js

icon.css
icon.js

button.css
button.js

input.css
input.js

Build

Bundles



project.css



project.js

Чтобы в результате сборки получить бандлы, необходимо определить:

- Список БЭМ-сущностей
- Зависимости между ними
- Порядок их подключения

Чтобы включить в сборку только необходимые БЭМ-сущности, нужно составить список блоков, элементов и модификаторов, используемых на странице. Такой список называется декларацией. Он позволяет избавиться от ненужного кода, который значительно увеличивает объем бандлов

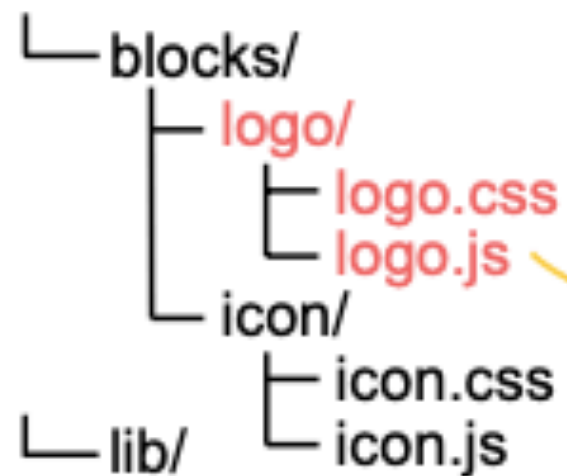
Инструмент сборки, основываясь на данных из списка, добавляет в бандлы только перечисленные БЭМ-сущности. Далее приведен пример сборки бандлов на основе декларации

FRONT-END

МЕТОДОЛОГИЯ БЭМ — СБОРКА — СПИСОК БЭМ-СУЩНОСТЕЙ

Source files

project/



logo.css
logo.js

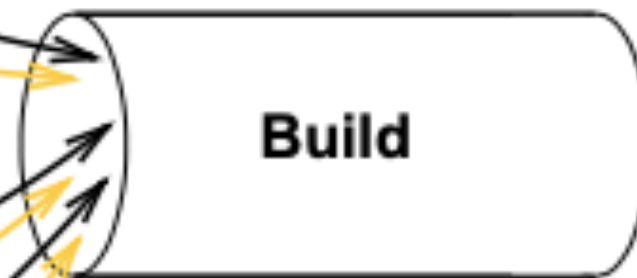
icon.css
icon.js

button.css
button.js

input.css
input.js

Declaration

button, logo, input



Bundles



project.css



project.js

В БЭМ блоки могут строиться на основании других блоков. Для этого необходимо указать зависимости от них. Зависимости позволяют избавиться от ненужного «Copy-Paste».

Инструмент сборки получает данные о зависимостях и добавляет все БЭМ-сущности, необходимые для реализации блока. Ниже приведен пример составного блока

search-form block

A diagram showing a 'search-form block' at the top, which is a horizontal rectangle divided into two parts: a white input field on the left and a yellow 'Search' button on the right. Below the input field is an 'input block' (a white rectangle), and below the button is a 'button block' (a white rectangle containing the text 'Button'). Two black arrows point upwards from the 'input block' and 'button block' respectively to the corresponding parts of the 'search-form block'.

input block

button block

FRONT-END

МЕТОДОЛОГИЯ БЭМ — СБОРКА — ПОРЯДОК ПОДКЛЮЧЕНИЯ

Порядок подключения БЭМ-сущностей в сборку зависит от:

- указанных зависимостей
- используемых уровней переопределения

Зависимости и порядок подключения БЭМ-сущностей в сборку

В БЭМ зависимости могут влиять на порядок подключения БЭМ-сущностей в сборку. Механизм подключения зависит от используемых DEPS-сущностей, которые по-разному влияют на приоритет подключения

Уровни переопределения и порядок подключения БЭМ-сущностей в сборку

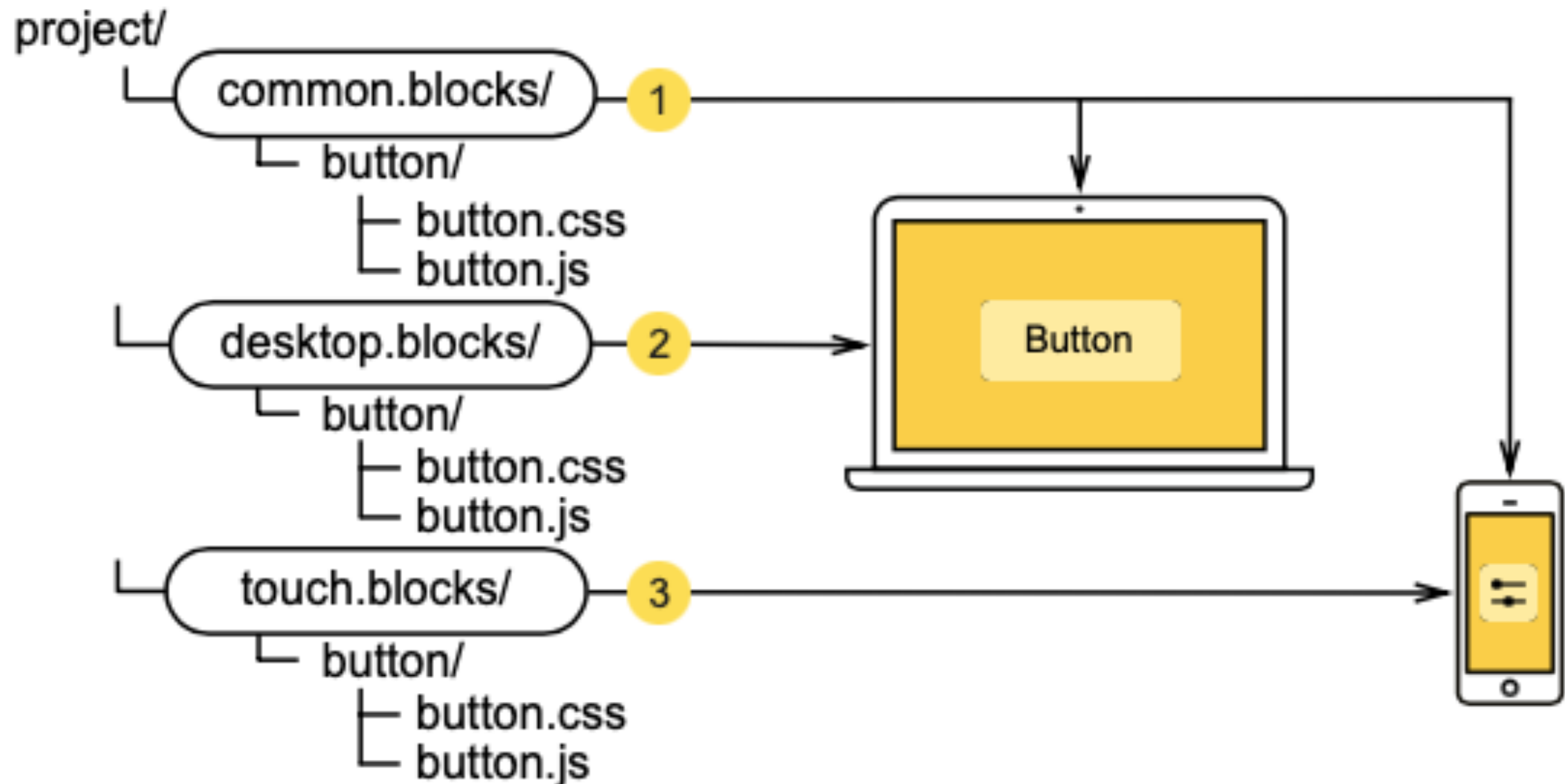
В БЭМ конечная реализация блока может быть разделена по разным уровням переопределения. Они позволяют изменять представление и поведение блоков для различных платформ. Каждый последующий уровень дополняет или перекрывает исходную реализацию блока. Поэтому важно, чтобы сначала в сборку попадала исходная реализация, а затем изменения со всех уровней переопределения

FRONT-END

МЕТОДОЛОГИЯ БЭМ — СБОРКА — ПОРЯДОК ПОДКЛЮЧЕНИЯ

Проект с уровнями переопределения: `common.blocks`, `desktop.blocks`, `touch.blocks`. Порядок сборки определен цифрами

Source files



FRONT-END

МЕТОДОЛОГИЯ БЭМ — СБОРКА — РЕЗУЛЬТАТ

В результате сборки можно получить файлы для:

- фрагмента страницы (например, `header.css` или `footer.css`)
- отдельной страницы (например, `hello.css` и `hello.js`)
- всего проекта (например, `project.css` и `project.js`)

При сборке отдельной страницы/проекта в результирующий код могут быть включены:

- все БЭМ-сущности с файловой структуры проекта (значительно увеличивает объем кода)
- только необходимые БЭМ-сущности

FRONT-END

МЕТОДОЛОГИЯ БЭМ — СБОРКА — РЕЗУЛЬТАТ

Файловая структура БЭМ-проекта до сборки:

```
blocks/                                # Директория, содержащая блоки

bundles/                               # Директория, содержащая результаты сборки (опционально)
  hello/                               # Директория страницы hello (создается вручную)
    hello.decl.js                      # Список БЭМ-сущностей, необходимых для страницы hello
```

Файловая структура БЭМ-проекта после сборки:

```
blocks/

bundles/
  hello/
    hello.decl.js
    hello.css      # Собранный CSS-файл страницы hello (бандл hello в технологии CSS)
    hello.js       # Собранный JS-файл страницы hello (бандл hello в технологии JS)
```

FRONT-END

МЕТОДОЛОГИЯ БЭМ — СБОРКА — ИНСТРУМЕНТЫ

БЭМ-методология не ограничивает выбор инструмента для сборки

В БЭМ-платформе используются следующие сборщики:

- ENB
- Gulp

FRONT-END

JS

FRONT-END

PWA

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Travel</title>
6      <meta http-equiv="X-UA-Compatible" content="IE=edge">
7      <meta name="viewport" content="width=device-width, initial-scale=1">
8      <meta name="description" content="Travel around the world">
9      <meta name="keywords" content="travel tour tourist">
10     <meta name="theme-color" content="#deb887">
11     <link rel="stylesheet" href="style.css">
12     <link rel="manifest" href="manifest.webmanifest">
13     <link rel="apple-touch-icon" href="icon-192x192.png">
14 </script>
15
16 if ('serviceWorker' in navigator) {
17     navigator.serviceWorker.register('sw.js')
18     .then(() => navigator.serviceWorker.ready.then((worker) => {
19         worker.sync.register('syncdata');
20     })))
21     .catch((err) => console.log(err));
22 }
23 </script>
24
25 </head>
26 <body> ... </body>
36 </html>
37
```


FRONT-END

PWA

manifest.webmanifest (travel-git) — Brackets

```
1 {
2   "name": "Touropoperator Travel",
3   "short_name": "Travel",
4   "theme_color": "#deb887",
5   "background_color": "#ffffff",
6   "display": "fullscreen",
7   "start_url": "https://victoriasoborova.github.io/travel-
8   git/index.html",
9   "icons": [
10     {
11       "src": "icon-192x192.png",
12       "sizes": "192x192",
13       "type": "image/png",
14       "purpose": "any maskable"
15     },
16     {
17       "src": "icon-256x256.png",
18       "sizes": "256x256",
19       "type": "image/png",
20       "purpose": "any maskable"
21     },
22     {
23       "src": "icon-384x384.png",
24       "sizes": "384x384",
25       "type": "image/png",
26       "purpose": "any maskable"
27     },
28     {
29       "src": "icon-512x512.png",
30       "sizes": "512x512",
31       "type": "image/png",
32       "purpose": "any maskable"
33     }
34   ]
35 }
```

sw.js (travel-git) — Brackets

```
1 ▼ self.addEventListener('install', (event) => {
2   console.log('Установлен');
3 });
4
5 ▼ self.addEventListener('activate', (event) => {
6   console.log('Активирован');
7 });
8
9 ▼ self.addEventListener('fetch', (event) => {
10   console.log('Происходит запрос на сервер');
11 });
```

BACK-END

PHP

- Обратная связь
- Комментарии
- Встраивание повторяющегося кода
`<?php require "footer.html"; ?>`

BACK-END

PHP — обратная связь

```
307 ▼ <form action="php/feedback.php" method="POST">
308     <input type="text" name="nam" placeholder="Name">
309     <input type="text" name="sub" placeholder="Subject">
310     <input type="text" name="ema" placeholder="E-mail">
311     <textarea name="mes" rows="9" placeholder="Message"></textarea>
312     <input type="submit" name="sen" value="SEND MESSAGE">
313 </form>
```

BACK-END

PHP — обратная связь

```
1  <?php
2
3  $errors = array();
4
5  if($_POST['nam'] == "") $errors[] = "the 'name' field is empty!";
6  if($_POST['sub'] == "") $errors[] = "the 'subject' field is empty!";
7  if($_POST['ema'] == "") $errors[] = "the 'email' field is empty!";
8  if($_POST['mes'] == "") $errors[] = "the 'message' field is empty!";
9
10
11  if(empty($errors)){
12      $mail_to = "v.v.soborova@gmail.com";
13      $subject = "Feedback";
14
15      $message .= "Name: " . $_POST['nam'] . "<br>";
16      $message .= "Subject: " . $_POST['sub'] . "<br>";
17      $message .= "E-mail: " . $_POST['ema'] . "<br>";
18      $message .= "Message: " . $_POST['mes'] . "<br>";
19
20
21
22      $headers .= 'MIME-Version: 1.0' . "\r\n";
23      $headers .= 'Content-type: text/html; charset=utf-8' . "\r\n";
24      $headers .= 'From:SHOPNO <root@cn56412.tw1.ru>' . "\r\n";
25
26      $send = mail($mail_to, $subject, $message, $headers);
27
28      if ($send == 'true') {
29
30          echo "<p style='color: green;'>Письмо отправлено</p><br><a href='../index.html'>назад</a>";
31      }
32  else {
33      echo "
34      <p style='color: red;'>Ошибка отправки сообщения!</p><br><a
35      href='../index.html'>назад</a>";
36  }
37  }
38  else{
39      $msg_box = "";
40
41      foreach($errors as $one_error){
42          $msg_box .= "<div style='color: red;'>$one_error</div><br>";
43      }
44  }
45  echo json_encode(array($msg_box));
46  ?>
47
```

DOCKER

APP

PLUGINS

command

```
cmd+N -- new file
cmd+O -- open file
cmd+S -- save file
cmd+W -- close file
shift+cmd+W -- close all files
cmd+Q -- close brackets

!+Tab -- new template
cmd+A -- all
cmd+C -- copy code
cmd+V -- paste code
cmd+D -- copy+paste code
cmd+E -- fast edit code
cmd+K -- documentation
cmd+/ -- blocks comment
esc    -- fast edit code

alt+cmd+P -- live preview
```

emmet

```
beautify
Autoprefixer
live preview
FTP Sync - клиент
Extract for Brackets (psd)
PSD Layer Viewer
Color Palette
Brackets Font
HTTP Server for Brackets
Quick Search
Autosave Files on Window Blur
Code overview
Document Toolbar
Brackets Git
Brackets SASS
```